We Asked an AI Database to Delete 100 Patients.

All 100 Were Still There.

We processed 100 erasure requests against the memory of a synthetic Australian clinic's AI receptionist, on the current release of a vector database thousands of AI products are built on, configured exactly as its quick-start configures it. Every check an operator can run said the patients were gone. Then we read the files. Measured, not guessed.

Wen-Chia “Belle” Chang · Agenvana

All data synthetic · all methods open-source and reproducible

THE ONE FINDING

One hundred patients asked to be erased, and we erased them with the one line of code the documentation gives you for exactly that.

We built the memory of an AI receptionist for an Australian dental clinic: 2,000 patient records, each one a free-text clinical note carrying a name, a date of birth, a phone number, an email address, a Medicare number and an address. Then we did what the law expects and what every tutorial demonstrates.

The system agreed they were gone. The record count dropped from 2,000 to 1,900. Fetching them by ID returned nothing. Searching with an erased patient's own clinical note returned three other people and never her. We restarted the database and checked again. Still gone.

Then we stopped asking the software and opened the file.

All 100 were still there. Bit for bit.

Every erased patient's vector sat intact in the index file on disk, next to the map that ties each one back to its patient ID. The file had not been modified by the deletion at all: its SHA-256 hash was identical before and after.

This issue is about the difference between deleted and gone, measured on the current release of a database that thousands of AI products are built on. Measured, not guessed.

EXECUTIVE SUMMARY

The one page to read

What we did. We loaded 2,000 synthetic patient records into ChromaDB 1.5.9, the current release of a widely used open-source vector database, configured exactly as its quick-start configures it. A vector database is the memory behind AI chatbots, AI receptionists and AI search: it stores a mathematical fingerprint of every document you feed it. We then processed 100 erasure requests using the documented delete call, confirmed through the application that the records were gone, and inspected the storage files directly. Every identifier is fabricated. No real person's data was used.

What we found.

- The delete satisfied every check an operator can make from inside the application, and left **100 of 100 erased vectors bitwise intact** in the index file. The index file was **byte-identical before and after** the deletion.
- Reading that file directly, with no credentials, no API and no database software, recovers **2,000 vectors from a collection that reports 1,900 records**. The 100 extra are the erased patients, still mapped to their patient IDs.
- **The vendor's own cleanup command makes this worse, not better.** `chroma vacuum` runs cleanly, reports success, and scrubs the erased patients out of the database file completely: names **100 to 0**, emails **100 to 0**, Medicare numbers **100 to 0**. It leaves **100 of 100 of their vectors** untouched in the index file beside it. An operator who runs it, then checks their work with a database query, sees a clean store and is wrong.
- **Deleting the entire knowledge base is not erasure either.** After dropping the whole collection, the storage directory survives with all 2,000 vectors still in it.
- A vector the system says was deleted is **exactly as useful for identifying its subject as one that was never deleted**. Holding the recovered vectors and nothing but a name, we picked the correct erased person out of 2,000 candidates on the first attempt 64 times in 100. Records that were never deleted scored the same or higher (70 to 79 in three runs), which is the point: erasure changed nothing about how identifiable these people are.

Why this matters to your business. Under Australian Privacy Principle 11.2, an organisation must take reasonable steps to destroy or de-identify personal information it no longer needs. The practical problem this report documents is not that the software is malicious. It is that **the software tells you the job is done, and the tooling you would use to verify it agrees.** A clinic that receives an erasure request, processes it correctly, runs the documented cleanup, and confirms with a database query has done everything a careful operator would think to do, and the patient's identifiable record is still on the disk, in the backups taken since, and in every copy of that directory that has ever been made. The gap is not between negligent and careful operators. It is between careful operators and the storage layer, and nothing in the standard toolchain shows it to you.

The good news, and it is real. This is a visibility problem before it is an engineering problem. Page 11 is a checklist you can work through this week: ask your vendor one specific question, look in one specific place, and treat the vector store as what it is, a copy of your customer data with its own lifecycle. Prevention exists and is buildable today; the honest state of the ecosystem is that almost nobody is doing it yet, and the first step is knowing the gap is there.

Credit where it is due. The underlying phenomenon was published first, in June 2026, by the authors of *Ghost Vectors* (arXiv:2606.18497), who tested Chroma, FAISS and Weaviate. Our contribution is narrower: we reproduce it on the current release with stock defaults, showing it is not fixed, and we test the cleanup paths a builder would actually reach for. That is where the sharpest finding in this report came from.

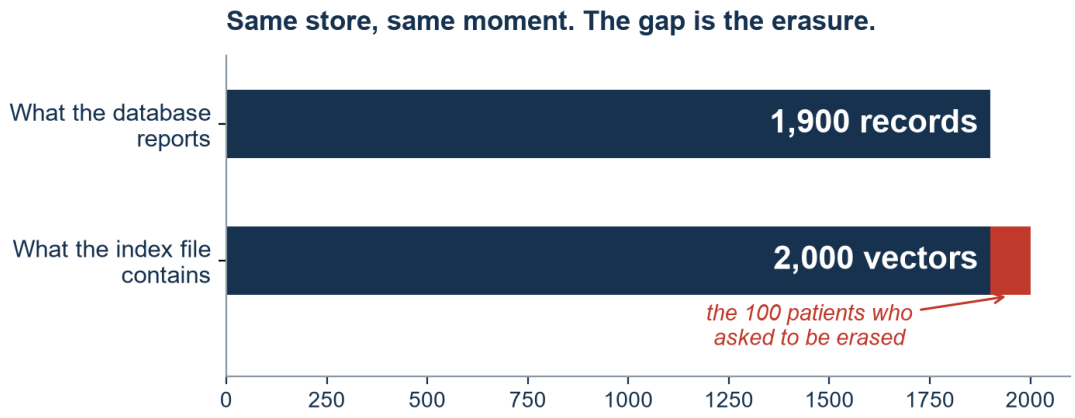
THE FINDING

What the application sees, and what is on the disk

Both tables below describe the same store at the same moment.

The check an operator can run	Result
collection.count() after the delete	1,900 (was 2,000)
collection.get(ids=[the 100 erased])	empty
Search using an erased patient's own clinical note	returns 3 other patients, never her
All of the above again after a full restart	still gone

What is actually in the files	Result
Index file (data_level0.bin) SHA-256, before vs after the delete	**identical: the delete never wrote to it**
Erased patients' vectors still present, bit for bit	**100 / 100**
Vectors recoverable by reading the raw index file, no database software	**2,000, for a collection reporting 1,900**
Erased patient IDs still in the segment's internal ID map	**100 / 100**
Erased patients' email addresses still byte-present in the database file	100 / 100
Erased patients' Medicare numbers still byte-present	100 / 100
... of those, returned by a query against any column of any table	0 / 100
Write-ahead-log rows still holding an erased record's text or vector	0 / 100



ChromaDB 1.5.9, stock defaults. The 2,000 vectors were recovered by reading data_level0.bin directly: no credentials, no API, no database software.

THE FINDING, CONTINUED

Two lines in those tables deserve emphasis, and they point in opposite directions.

The vendor fixed a real problem, and should be credited for it. Chroma's write-ahead log used to retain the full text and embedding of deleted documents (chroma-core issue #3793, February 2025, since closed). In 1.5.9, automatic purging is on by default, and we measured zero surviving records there: 0 of 100. The 2025 behaviour does not reproduce at our store size. We went looking for it and it was gone, which is what a fixed bug looks like. One nuance surfaced upstream two days before publication (chroma-core issue #7659, 29 August 2026): the automatic purge waits on the index's persistence threshold, roughly a thousand operations, so a small and quiet store retains a deleted record's text in the log until enough further writes occur. Our 2,000-record store is well past that threshold, which is why we measured zero; a 50-record chatbot memory would still be holding its deletions.

And we have to be precise about where the identifiers do survive, because the obvious explanation is wrong and we tested it. The erased patients' emails and Medicare numbers are byte-present in the database file, and no query reaches them. It would be natural to assume this is the familiar “deleted rows linger in free space” story. It is not. We walked the database's own free-page list and mapped every occurrence: **not one of the 100 sits only in freed pages. 53 of 100 sit entirely in pages that are still in use**, and 47 have copies in both. The honest claim is the narrow one, and it is the one that matters to an operator: **the identifiers survive in the file, and no query returns them.** That is precisely why checking your work with SQL shows you a clean store.

THE FINDING, CONTINUED

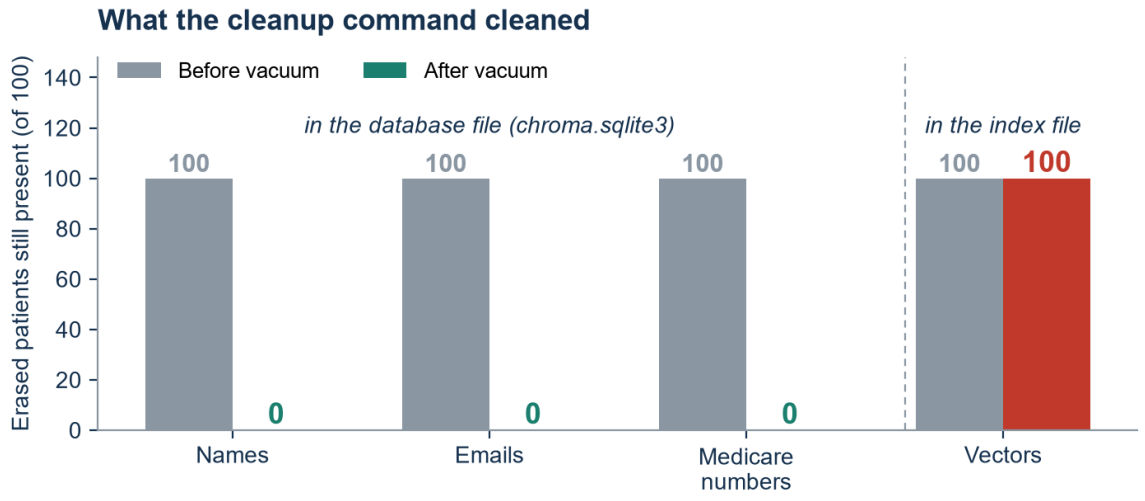
The three things a builder would try, and what each one actually does

What we tried	Erased vectors still intact	Erased identifiers still in the database file
Keep using the store (1,500 further writes, well past the point where the index rewrites itself)	**100 / 100**	100 / 100
chroma vacuum --path <store> --force, the documented cleanup, exit code 0	**100 / 100**	0 / 100
Drop the whole collection (delete_collection())	**100 / 100**	roughly 90 / 100

The middle row is the sharpest thing in this data set. The official maintenance command runs cleanly and reports success:

```
Vacuum complete!
Database size reduced by 2.2MiB
```

It does exactly what it says. The database file shrinks from about 6.7 MB to 4.38 MB, the free-page count goes from 496 to zero, and every erased patient's name, email address and Medicare number is scrubbed out of it: 100 to 0 on all three, measured after the command ran. An operator who runs that command has every reason to believe they have cleaned up. **The one file that still holds the erased people is the one the command does not rewrite.** All 100 vectors, and the ID map naming them, are exactly where they were.



chroma vacuum ran cleanly (exit 0) and shrank the database file. The command succeeded. It cleaned the file that was not the problem.

And dropping the entire knowledge base is not erasure either. After `delete_collection()`, the storage directory is still there, the 3.35 MB index file is still fully populated, all 2,000 vectors are still readable, and the ID map is still next to them. Deleting the collection removes the collection, not the data.

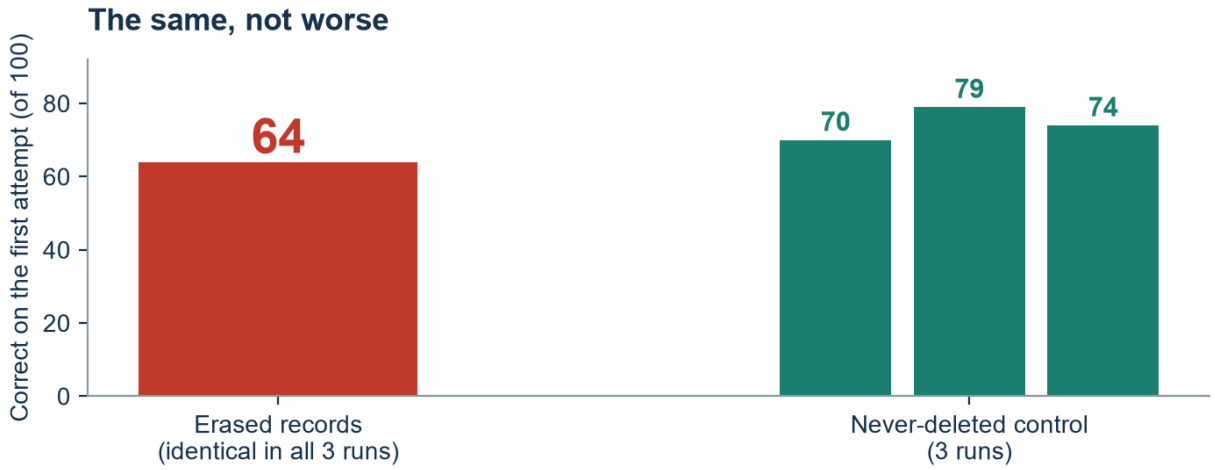
THE FINDING, CONTINUED

What a leftover vector is worth to someone who has it

We did not attempt to reconstruct text from any vector. That leg of the argument rests entirely on published research, cited in the consequences section, and we are careful not to blur the two. We measured something simpler and harder to dispute: **linkage.** Given the recovered vectors and nothing but a person's name, can you pick that person's supposedly deleted vector out of the pile of 2,000?

The attacker knows	Correct on the first attempt	Within the top 5
A name, erased records	64 / 100	83 / 100
A name and a date of birth, erased records	54 / 100	75 / 100
A name, records that were never deleted (control)	**70 to 79 / 100**	**84 to 93 / 100**

The control row is the important one, and it cuts against us. We ran the identical probe against 100 patients who were never deleted and whose notes are still live in the store. They scored higher than the erased records, in every run. So this measurement does **not** show that erasure makes people findable. The mechanism is ordinary retrieval: each patient's name is written inside the note that was embedded, and our probe uses the same model the store used.



Name-only linkage against 2,000 candidates, same probe both cohorts.
The never-deleted control scored higher, so this shows erased records are no less identifiable than live ones, not that erasure exposed anyone.

What it does support, stated narrowly, is the claim the rest of this report needs: **a vector the system says was deleted remains exactly as useful for identifying its subject as one that was never deleted.** Erasure changed nothing about the identifiability of these 100 people. Any stronger reading of the 64 is unsupported, and the method section says so again.

HOW WE MEASURED

Method, and everything wrong with it, stated by us first

- **The fixture.** 2,000 synthetic patient records, each a free-text clinical note containing a name, date of birth, phone, email, Medicare number and address, written the way real front-desk software writes them. Every identifier is fabricated and belongs to no one.
- **The system under test.** ChromaDB 1.5.9, Python 3.13.5, a local persistent client, all defaults (l2 index space, sync threshold 1,000, the default all-MiniLM-L6-v2 embedding model at 384 dimensions). Deterministic seed. The script, the environment file and the raw results for all three runs are published, and the rerun command is in the appendix.
- **Store size is not arbitrary.** Chroma writes its index to disk only after 1,000 writes, so a 50-document demo never creates the file we examined. 2,000 records is the smallest realistic size at which the on-disk index exists. If someone tries to reproduce this on a toy store and reports that it does not happen, this is why.
- **We ran it three times and kept all three sets of raw results.** Every headline number in this report was identical in all three: the 1,900 count, the byte-identical index comparison, 100 of 100 vectors, 100 of 100 identifiers on disk, 0 of 100 reachable by query, 0 of 100 in the log, the 2,000 blind-recovered vectors, and the 64. Exactly four figures varied, and we quote each one loosely or not at all: the control cohort (70, 79, 74), the residue after dropping the collection (92, 91, 89, quoted as “roughly 90”), the size of the database file before the cleanup command ran (6.68, 6.66 and 6.65 MB, quoted as “about 6.7 MB”), and one internal storage slot number, which we do not quote anywhere for that reason.
- **The index comparison is within a single run, deliberately.** Building this kind of index is not deterministic, so two runs produce different files. The claim “the file was not modified” is a before-and-after hash comparison inside one run, which is the only form of that claim that means anything.

Limitations that cut against our own numbers, in plain terms:

– **One database, one embedding model, running locally.** We do not claim these numbers for Pinecone, Qdrant, Weaviate or a managed cloud deployment. The *Ghost Vectors* paper covers FAISS and Weaviate; we cite it rather than extrapolating from our own store.

– **We did not reconstruct text from a vector.** Our numbers are linkage, not inversion. The “a vector can become text again” claim is the literature's, with citations, not ours.

– **Every patient in our fixture has a unique name**, which makes name-only linkage cleaner than reality. Real patient lists repeat names, and the 64 would be lower.

– **Our re-identification probe uses the same embedding model the store used**, and the patient's name is written inside the note that was embedded. Someone without the model would do worse. This is why the never-deleted control scored higher than the erased records, and why the 64 measures the retrievability of a name-bearing note rather than any property of erasure.

What we got wrong on the way, since the method is the product. Our first pass at this data claimed that the erased identifiers were lingering in freed database pages. We had inferred that from three hand-picked probes without walking the actual free-page list. When we walked it, the claim was refuted by our own measurement, and the narrower true claim replaced it. Our first pass also claimed the write-ahead log still held deleted records, which was a misreading of deletion markers as surviving data, and it is now a credit to the vendor instead. A third claim, about how much re-identification proves, was cut down by a control cohort we added specifically to attack it.

WHAT IT MEANS

Consequence, by sector

The honest frame first. We could not find a publicly documented breach of a vector database, and that should worry you rather than comfort you. Breach reports never say “the attacker took the embedding store”. No standard forensic tooling looks there. Most organisations do not know their chatbot has one. Absence of headlines is absence of instruments, not absence of events. What can be documented is that these stores are attackable (a critical authentication bypass in Milvus, CVE-2026-26190, CVSS 9.8, was published this year), that their contents are recoverable (the research below), and that the residue this report measured needs no exploit at all. It is a file. Anyone who can read the directory can read the vectors, and that includes a backup, a decommissioned laptop, a departing contractor, and every copy of the storage folder that has ever been made.

A vector is not anonymous, and this is the part most operators have not internalised. Published research recovers text from embeddings: 92% of 32-token inputs reconstructed exactly (Morris et al., EMNLP 2023), with the same technique recovering full names from a clinical-notes dataset, independently replicated in 2025. The *Ghost Vectors* authors, working specifically on soft-deleted embeddings in vector databases including Chroma, recovered 25.5% of exact person names and 46.4% of geographic locations on a general dataset, and 100% of patient age and gender markers on a synthetic health dataset. We did not re-run any of that. We measured that the vectors those techniques operate on are still sitting in the file after the person asked to be forgotten.

Health

A dental or medical practice running an AI receptionist has a patient's name, date of birth, contact details and Medicare number inside the model's memory, because that is what the notes contain. When that patient asks to be erased, the practice does the delete, and every check available to the practice says it worked. The Medicare number is a government identifier with specific handling rules, the clinical detail is sensitive information, and the record is still on the disk and in every backup taken since. The consequence is not a fine on day one. It is that the practice has told a patient something that is not true, in writing, and cannot discover the error with any tool it owns.

Legal

A law firm's AI memory holds something a clinic's does not: the fact of the relationship. In a family-law, criminal or insolvency matter, confirmation that a person is a client is itself the sensitive fact, and it is exactly what a leftover vector still supports. Firms also carry an obligation to destroy or return client material at the end of a retainer. A “deleted” matter that is still reconstructible from an index file is a version of that obligation that no file-destruction policy covers, because the policy was written for paper and for documents, not for the derived copy the AI system made.

Financial services and brokers

Loan files are identifier-dense by design, and the harm from leaked identifiers arrives later, laundered through fraud. A scam is credible in exact proportion to the accurate personal data behind it: the right name, the right lender, the right account format, a plausible amount. Data leaks do not hurt people directly. They arm the scam that does, months later, wearing someone else's logo, and nobody ever traces it back to a broker's AI assistant. When that wave reaches your customers, the investigation walks backwards, and “we deleted them, the system said so” is a sentence that will not survive the first look at the file.

Why I believe the file and not the message

Years ago, I told an export trading company to delete the customer and supplier files from a desktop that was being repurposed, and to format the disk after; they did what I asked. I only caught the leftovers because I happened to look: the recycle bin still held every file, and another copy sat in the backups on their office server, where I had no access to verify anything. They believed the job was done, and every check they knew how to run agreed with them.

The compliance sentence, and the reason this report exists. The regulator's own language is about destruction. The OAIC's December 2025 compliance sweep of around 60 organisations assessed them against transparency obligations and used the word “destroyed” in its own description of what a privacy policy must explain, with penalties up to $66,000 for the notices in play. New automated-decision transparency obligations commence 10 December 2026. Against that clock, the finding in this report is uncomfortable in a specific way: **an operator who processes an erasure request correctly and runs the vendor's documented cleanup has a reasonable belief that they have destroyed the data, and is wrong.** Reasonable belief is not the same as reasonable steps, and the difference is currently invisible to everyone who is not reading raw files.

WHAT TO DO

Seven steps: doable this week, none of them require buying anything

1. **Ask your vendor the one question that has a file-level answer.** Not “is the data deleted”, which every vendor can answer yes to in good faith. Ask: **“When a deletion is processed, which files are rewritten, and does that include the vector index?”** The answer to that question is checkable, and the difference between the two questions is this entire report.
2. **Look in the directory.** If you run a vector store yourself, the index file is on your disk right now. Compare the record count your application reports against the size of that index file over time. A store that reports fewer records while its index file never shrinks is the pattern documented here. This is a five-minute check that no dashboard performs for you.
3. **Do not trust a successful cleanup command as evidence of erasure.** This is the finding that costs an operator the most, because it is the one that produces a false record of compliance. `chroma vacuum` exits 0, prints a success message, genuinely shrinks the database, and leaves every vector in place. If your evidence that a person was erased is “the maintenance job ran clean”, you do not have evidence.
4. **Verify erasure from outside the system that performed it.** The only check that means anything is one that does not ask the application whether it deleted something. Read the storage, and confirm the subject is not recoverable from it. If nobody in your organisation can do that, that is the finding, and it is worth knowing before someone asks you to prove it.
5. **Treat the vector store as a copy of your customer database, because it is one.** It belongs in your data inventory, your retention schedule, your access controls, your patching cycle and your breach-response plan. Most organisations we talk to have it in none of them, usually because it arrived as a feature of something else rather than as a database somebody chose to run.
6. **Bring backups into the erasure question.** A record that was on disk when a backup was taken is in that backup, and an erasure that only touches live storage was never complete. This is the question that turns a five-minute fix into an architecture conversation, which is why it belongs at the end of this list rather than the start.
7. **Design so that erasure is possible at all.** The structural answer is to keep the sensitive values out of what gets embedded and to hold them somewhere revocable, so that erasure is an act you perform once and can prove, rather than a delete you hope propagated to every copy. The industry is not there yet. Knowing that the delete call is not that mechanism is the first honest step.

APPENDIX

Methodology detail, reproduction and sources

Reproduce it. The script, the pinned environment and the raw results for all three runs are published with the report. From the repository root:

```
python -m venv reports/tools/chromaenv
reports/tools/chromaenv/Scripts/python.exe -m pip install \
    -r reports/tools/requirements-chroma.txt
reports/tools/chromaenv/Scripts/python.exe reports/findings/src/chroma_erasure.py
```

(On macOS, substitute `reports/tools/chromaenv/bin/python`.) Verified on chromadb 1.5.9, Python 3.13.5, Windows, with a fixed seed.

How the index file is laid out, for anyone who wants to check the recovery claim themselves: the file is 3,352,000 bytes, which is 2,000 slots of 1,676 bytes each. Every slot holds 132 bytes of graph structure, then 384 floating-point values (1,536 bytes), then an 8-byte label that ties the slot to a record ID. Reading it requires no database software, no credentials and no API. It requires the file.

Texture from the run, unedited. Erased patient PT-00049, “Marguerite Kaur”, recovered from the index file after the API confirmed her deletion, with a cosine similarity of 1.0 to her original vector. Her first six values, byte-identical across all three runs:

```
[-0.11747640371322632, 0.01099690143018961, 0.04241495579481125,
 -0.004311075899749994, -0.1106862723827362, -0.002682364545762539]
```

Sources. Ghost Vectors: Soft-Deleted Embeddings Remain Reconstructible in HNSW Vector Databases, arXiv:2606.18497 · Text Embeddings Reveal (Almost) As Much As Text, arXiv:2310.06816 · Vec2Text reproducibility study, arXiv:2507.07700 · ALGEN, arXiv:2502.11308 · chroma-core issue #3793 and CIP-07102024 · chroma-core issue #7659 (WAL retention below the sync threshold, 29 August 2026) · CVE-2026-26190 · OAIC APP Guidelines chapter 11 (APP 11.2) · OAIC privacy compliance sweep, 09 December 2025 · GDPR Article 17. All primary-source verified with access dates in the findings package.

Responsible disclosure

We reported this to the ChromaDB maintainers on **31 August 2026**, before publication, as issue **#7662** in `chroma-core/chroma`, with the script and raw results needed to reproduce it.

About

The AI Data Security Report is a monthly, methods-open report on where AI pipelines actually leak data. Measured, not guessed. It never names a villain it hasn't benchmarked and never states a number it can't reproduce. Author: **Wen-Chia “Belle” Chang**, Agenvana. The author's work on data-layer privacy for AI systems is patent pending. Contact: via LinkedIn (Belle Chang) or github.com/Agenvana.

Issue #1 (July 2026), *“Your AI's Privacy Filter Speaks American”*: we benchmarked what the standard PII filter actually catches. Issue #3 (September 2026): the next measurement is chosen by what this issue's readers ask.